

Clustering Assignment- F1 Clustering Project

Roe Amsalem, Oren Yehezkel, Ido Brener

1 Introduction:

Question 1.1: Why is clustering considered a form of unsupervised learning?

What information is missing compared to supervised learning?

Clustering is considered a form of unsupervised learning because the algorithm operates on unlabelled data. It aims to discover inherent structures, patterns, or groupings within the dataset without any prior guidance. In contrast to supervised learning, clustering completely lacks "target" variables or ground-truth labels. The model does not have a predefined correct answer or category to map the input features to; instead, it must independently identify relationships based solely on the mathematical similarity or distance between the data points.

Question 1.2: What are the input and output of a clustering algorithm?

Input: The input consists of a dataset containing a set of features (multidimensional vectors) for each observation. This data is entirely unlabelled, meaning there is no target variable associated with the observations.

Output: The primary output is a set of cluster assignments. The algorithm returns a distinct label or identifier for each observation, mapping every original data point to a specific generated group (cluster) based on the learned patterns. Secondary outputs may include cluster centers (centroids) or density boundaries, depending on the specific algorithm used.

3 Similarity and Distance:

Question 3.1: Give an example of a situation in which Euclidean distance may not be the most natural notion of similarity.

An example of this would be text document clustering in Natural Language Processing (NLP). When text documents are represented as word frequency vectors (e.g., Bag of Words or TF-IDF), Euclidean distance is heavily distorted by the length of the documents. For instance, a short article and a massive book covering the exact same topic will have a very large Euclidean distance simply due to the magnitude of their word counts, falsely suggesting they are dissimilar. In this scenario, **Cosine Similarity** is a much more natural and effective metric, as it evaluates the angle between the vectors (the context or topic distribution) completely independently of their magnitude (the document length).

Another practical physical example is navigating a structured city grid. Euclidean distance calculates the "as-the-crow-flies" straight line passing through physical obstacles (like buildings), which is completely irrelevant for driving or walking. In this case, Manhattan Distance (L1 norm) is the natural and realistic metric.

4 K-means Clustering:

Question 4.1: In the assignment step, what is fixed and what is being computed?

In the assignment step of the K-Means algorithm, the cluster centroids are fixed. The algorithm computes the distance (typically Euclidean) between every single data point and each of the fixed centroids. Based on these computed distances, it updates the cluster assignments by allocating each data point to the cluster with the nearest centroid.

Question 4.2: In the update step, what is fixed and what is being computed?

In the update step, the cluster assignments (the labels of the data points) are fixed. The algorithm computes the new coordinates for each cluster's centroid. It does this by calculating the mean (average) of all the feature vectors of the data points that are currently assigned to that specific cluster.

Question 4.3: Why does K-means tend to produce clusters that are approximately spherical?

K-Means tends to produce spherical clusters because it relies on Euclidean distance to assign points to the nearest centroid. Euclidean distance radiates equally in all directions from a central point. By minimizing the sum of squared distances to the center, the algorithm implicitly assumes that the underlying clusters are convex, roughly circular (or spherical in higher dimensions), and have similar variance.

Question 4.4: Give an example of a dataset where K-means is expected to perform poorly, and explain why.

For example where K-Means fails is a dataset shaped like two concentric circles (like a donut). Because K-Means uses Euclidean distance and explicitly tries to build spherical clusters, it cannot capture the ring structure. Instead of intelligently separating the inner circle from the outer ring, it will just cut the dataset straight down the middle.

Question 4.5: Why is K-means not guaranteed to find the globally optimal clustering?

K-Means is not guaranteed to find the global optimum because its objective function (minimizing the Within-Cluster Sum of Squares) is non-convex, meaning it contains multiple local minima. The algorithm relies on an iterative, greedy optimization process that strictly moves downhill from its starting point. Therefore, depending on where the initial centroids are placed, the algorithm can easily get trapped in a suboptimal local minimum rather than the true global minimum.

Question 4.6: Compare the computed centroids with your manual calculation.

The centroids computed by the algorithm perfectly match our manual calculations. For the first cluster, the center is the exact average of the first three data points (27, 32.33), and for the second cluster, it is the average of the last three points (70, 79.33).

Question 4.7: Run the code several times with different random states. Do the results always stay the same?

Yes, for this specific dataset provided in the exercise, the results stay exactly the same regardless of the random state. The two groups of points are placed so far apart from each other that the algorithm effortlessly finds the true global minimum every single time without getting stuck.

Question 4.8: If the answer to the previous question is yes, construct a small new dataset for which running K-Means with different random states may lead to different clustering results. Explain why the initialization can matter in your example.

We can construct a simple dataset of 4 points arranged in a perfect square: $[0,0]$, $[0,2]$, $[2,0]$, and $[2,2]$. If we run the algorithm with $k=2$, the initialization matters a lot. Depending on where the initial random centroids land, K-Means might group the top two points together and the bottom two together, or it might group the left two points together and the right two together. Since both options are mathematically valid local minima, the random starting state completely dictates the final outcome.

5 Hierarchical Clustering

Question 5.1: How does hierarchical clustering differ conceptually from K-means?

Conceptually, K-Means is a "flat" clustering algorithm that requires the user to pre-define the exact number of clusters (k) in advance, and it aims to partition the data into exactly that many spherical groups. In contrast, hierarchical clustering builds a multi-level, tree-like structure of nested clusters. It does not require a pre-defined number of clusters. Instead, it starts with every individual data point as its own cluster and iteratively merges the closest pairs together until all points are grouped into one overarching cluster. The user can then choose the final number of clusters by virtually "cutting" the tree at their desired level.

Question 5.2: What is the difference between single linkage and complete linkage?

The difference lies in how the mathematical distance between two clusters is calculated when deciding which clusters to merge. Single linkage calculates the distance based on the minimum distance between any single point in the first cluster and any single point in the second cluster (nearest neighbors). This approach tends to produce long, chain-like, elongated clusters. Complete linkage calculates the distance based on the maximum distance between any point in the first cluster and any point in the second cluster (furthest neighbors). This approach ensures all points within the newly merged cluster are close to each other, producing highly compact and spherical clusters.

Question 5.3: In what sense can average linkage be viewed as a compromise between single and complete linkage?

Average linkage calculates the distance between two clusters by taking the average of the distances between all possible pairs of points (one from the first cluster, one from the second). It acts as a compromise because it avoids the extremes of the other two methods. It prevents the "chaining" effect caused by the overly optimistic single linkage (which only considers the closest single pair), while also being significantly less sensitive to extreme outliers than the overly pessimistic complete linkage (which is dictated entirely by the single most distant pair).

Question 5.4: What information does a dendrogram provide?

A dendrogram is a tree-like visual diagram that records the entire hierarchical sequence of cluster merges in the dataset. The horizontal axis represents the individual data points or subgroups, and the vertical axis (height) represents the mathematical distance or dissimilarity between clusters at the exact moment they were merged. By looking at a dendrogram, you can clearly see the structure of the data, understand which points

are most similar, and visually determine a natural cutoff point to achieve the optimal number of discrete clusters.

6 DBSCAN

Question 6.1: What is the main idea behind DBSCAN?

The main idea behind DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is to group together points that are closely packed together in high-density regions, while explicitly marking isolated points in low-density regions as outliers (noise). Unlike K-Means, which minimizes distances to a central point and assumes spherical clusters, DBSCAN traverses the data space to find areas where a minimum number of points exist within a specified radius. This allows it to discover clusters of arbitrary, non-convex shapes without needing to know the number of clusters in advance.

Question 6.2: Give an example of a dataset where DBSCAN is expected to perform well.

DBSCAN excels in datasets containing clusters with arbitrary, complex geometries embedded in spatial noise. A classic example is a "two-moons" dataset (two interlocking crescent shapes) or concentric rings. While K-Means would fail by drawing a straight line through these shapes, DBSCAN successfully identifies the continuous dense paths of the crescents/rings and accurately classifies any scattered background points as noise.

Question 6.3: Give an example of a dataset where DBSCAN may fail, and explain why.

DBSCAN performs poorly on datasets with varying densities. For example, consider a dataset containing two clusters: one highly compressed and tightly packed, and another naturally spread out and sparse. Because standard DBSCAN relies on a single global 'eps' (radius) parameter, it cannot adapt. If you set the radius small enough to resolve the dense cluster, the sparse cluster points will be falsely labeled as noise. If you set the radius large enough to capture the sparse cluster, the dense cluster might be erroneously merged with surrounding noise or neighboring clusters.

Question 6.6: Which point is labeled as noise in this example?

In the provided coding exercise, the point [8.0, 1.0] is labeled as noise (usually denoted by the label -1 in scikit-learn). The first three points form one dense group, and the next three form a second dense group, both satisfying the minimum sample and epsilon thresholds. The point [8.0, 1.0] is spatially isolated and too far away from any other points to meet the required density threshold of $\text{eps}=0.5$.

Question 6.7: Change the value of eps. How do the labels change?

Changing the 'eps' (epsilon) parameter drastically alters the clustering structure. If we significantly decrease 'eps' (e.g., $\text{eps} = 0.01$), the search radius becomes too small for any point to find the required 3 neighbors, resulting in the algorithm labeling every single point in the dataset as noise (-1). Conversely, if we significantly increase 'eps' (e.g., $\text{eps} = 10.0$), the search radius becomes so large that all points fall into each other's neighborhoods, merging the entire dataset into one single massive cluster (label 0).

9 The Curse of Dimensionality

Question 9.1: Why does the curse of dimensionality affect both K-means and DBSCAN?

The "curse of dimensionality" fundamentally disrupts the concept of spatial distance, which is the core metric for both K-Means and DBSCAN. As the number of dimensions in a dataset increases, the volume of the feature space expands exponentially. In such high-dimensional spaces, the mathematical difference between the distance to the nearest neighbor and the distance to the farthest neighbor converges. Essentially, all data points appear to be almost equidistant from one another. For K-Means, this is highly problematic because the algorithm minimizes the within-cluster sum of squares based on distance to centroids. If all points are roughly the same distance apart, the centroids lose their significance. For DBSCAN, the algorithm defines clusters based on local density. In high dimensions, the concept of a "dense neighborhood" evaporates because points are so sparsely distributed that almost everything is classified as noise, unless a massive radius is used, which indiscriminately groups unrelated points together.

10 Mini Project: Exploring Clustering Algorithms

Question 10.1: For each of your chosen clustering algorithms: apply it to a small part of your dataset, and discuss how parameter choices affect the results.

We applied K-Means and DBSCAN to our processed Formula 1 telemetry dataset (comprising 1,109 individual cornering events). For K-Means, we experimented with the 'n_clusters' parameter using the Elbow Method on the raw corner data. Setting n_clusters=3 provided a logical mathematical division, allowing the algorithm to categorize all 1,109 events into three distinct "cornering styles". For DBSCAN, we tuned the 'eps' (radius) and 'min_samples' parameters. However, we found that DBSCAN struggled significantly with our dataset. Without dimensionality reduction, using an eps of 0.7 resulted in the algorithm classifying a massive portion of the cornering events as pure noise (-1) due to the curse of dimensionality across the 12 telemetry features. Even when adjusting parameters, the continuous nature of F1 telemetry data—which lacks completely separated, isolated dense pockets of data points—made it impossible for DBSCAN to form meaningful groupings.

Question 10.2: A practical study should include changing the number of clusters, parameters, visualizing the resulting clusters, calculating the silhouette coefficient, and discussing how the clustering changes.

To deeply evaluate our data, we developed a sophisticated two-step clustering pipeline and compared two different engineering approaches:

The Two-Step Pipeline Concept: Instead of simply averaging driver telemetry, our pipeline first clusters the 1,109 individual corners into 3 "Cornering Styles" using K-Means. Then, we construct a "Driver Profile" for each driver by calculating the percentage distribution of how often they utilize each of the 3 corner styles. Finally, we run a second K-Means algorithm (also with k=3) on these profiles to categorize the drivers into 3 distinct groups based on their stylistic tendencies.

Approach A: No PCA (Clustering on Raw Features) In this approach, the initial K-Means clustering of the corners was performed directly on the full 12-dimensional feature space. The resulting driver profiles were then clustered into 3 driver groups. We visualized the final results by projecting the 3-dimensional driver

profiles into a 2D scatter plot using PCA. This method relied strictly on the raw, uncompressed telemetry metrics to define the baseline corner styles.

Approach B: PCA First (Dimensionality Reduction) In the alternative approach, we applied PCA to the initial corner data (retaining 95% of the variance) to compress the features and eliminate noise. We then ran the first K-Means ($k=3$) on these principal components to define the corner styles. The rest of the pipeline—building the cross-tabulated driver profiles and clustering the drivers into 3 groups—remained identical.

Conclusion: Changing the pipeline from "No PCA" to "PCA First" fundamentally altered the underlying definition of the "Cornering Styles," which directly affected how the drivers were ultimately grouped. While both pipelines utilized a $k=3$ configuration for the final driver clustering, Approach B (PCA First) allowed the initial K-Means algorithm to bypass the noise and the curse of dimensionality during the critical corner-classification step. This demonstrated that performing dimensionality reduction prior to clustering complex, continuous telemetry data can significantly refine the structural logic of the final driver profiles.